

Zinterpreter

Un interprete scritto in C per **Zinter**, un linguaggio di scripting personalizzato progettato da zero.

Perché esiste

Zinterpreter nasce come progetto didattico e sperimentale: costruire un linguaggio di scripting completo partendo da zero, senza dipendenze esterne, in C puro. L'obiettivo non è competere con linguaggi esistenti, ma capire dall'interno come funziona un interprete — dalla lettura del file sorgente fino all'esecuzione istruzione per istruzione.

Il progetto copre l'intera filiera:

- lettura e pulizia del codice sorgente
 - tokenizzazione e formattazione in righe eseguibili
 - costruzione dello stato (funzioni, blocchi, scope)
 - esecuzione tramite un parser lineare con gestione di variabili, array, matrici e funzioni
 - una VM (VM) che isola tutto lo stato del programma in un'unica struct
-

Struttura del progetto

```
Zinterpreter/  
├─ Zinterpreter.c      # sorgente principale (tutto in un file)  
├─ *.Zinter           # file sorgente Zinter da eseguire  
└─ *.Zlib             # librerie importabili
```

Come compilare ed eseguire

```
gcc Zinterpreter.c -o Zinterpreter  
  
./Zinterpreter -dt programma.Zinter  
./Zinterpreter -df programma.Zinter  
./Zinterpreter -dt programma.Zinter -libreria.Zlib
```

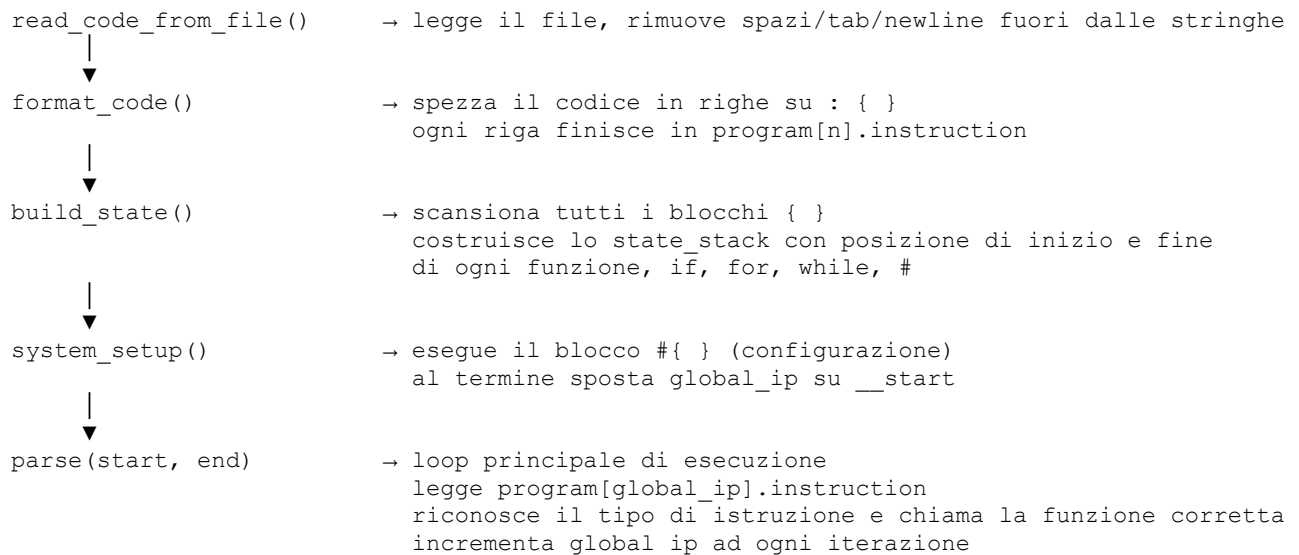
Se viene avviato senza argomenti, esegue automaticamente una batteria di test interni (`run_test()`).

Argomenti da riga di comando

Argomento	Significato
-dt	debug true — stampa tutto il flusso interno
-df	debug false — esecuzione silenziosa
file.Zinter	file sorgente Zinter da eseguire (obbligatorio)
-libreria.Zlib	libreria da includere (il - iniziale è parte della sintassi)

Flusso di esecuzione

```
file .Zinter  
  |  
  ▼
```



La VM

Tutta la memoria del programma Zinter è contenuta in una singola struct globale `vm`:

```
VM vm;
```

Contiene variabili `int/float/char`, array, matrici, registri, il programma formattato, lo stack degli stati e l'Instruction pointer (`global_ip`). Ogni campo è accessibile tramite macro:

```
#define variable      vm.variable  
#define variable_count vm.variable_count  
// ... ecc
```

Questo design permette in futuro di avere più istanze dell'interprete in parallelo semplicemente dichiarando più `vm`.

Sintassi Zinter

Regole generali

- ogni istruzione termina con :
- gli spazi sono ignorati **tranne** all'interno di "stringhe"
- il punto di ingresso del programma è `__start`
- i commenti iniziano con `//`

Tipi

Prefisso	Tipo
i	integer
l	float
c	char (solo variabili)
s	array di char / stringa
n	numero intero immediato
k	carattere immediato

Sintassi estesa (token)

I dati possono essere referenziati in forma esplicita con la notazione `&tipo&nome&`:

<code>&i&nome&</code>	→ variabile intera
<code>&l&nome&</code>	→ variabile float
<code>&c&nome&</code>	→ variabile char
<code>&s[idx]&nome&</code>	→ elemento idx di un array char
<code>&i[idx]&nome&</code>	→ elemento idx di un array int
<code>&i[r][c]&nome&</code>	→ elemento [r][c] di una matrice int
<code>&n&42&</code>	→ numero immediato 42
<code>&k&A&</code>	→ carattere immediato 'A'

In alternativa, si può usare il nome direttamente e l'interprete deduce il tipo automaticamente tramite `is_what()`.

Dichiarazioni

```
int_ &i&nome&:           // variabile int
int_ &i[10]&nome&:        // array int di 10 celle
int_ &i[5][5]&nome&:      // matrice int 5x5

char_ &c&nome&:          // variabile char
char_ &s[20]&nome&:       // array char (stringa) di 20 celle
char_ &s[3][3]&nome&:     // matrice char 3x3
```

Assegnazione

```
var1 = var2:             // copia var2 in var1
var1 = 42:               // assegna numero immediato
[0]arr = var1:           // arr[0] = var1
[0]arr = [1]arr2:        // arr[0] = arr2[1]
[0][1]matr = var1:       // matr[0][1] = var1
var1 = var2 + var3:       // somma (supporta + - * /)
var1 = __funzione(arg):   // assegna il valore restituito da una funzione
```

Forma alternativa esplicita:

```
set_to_variable_ nome,i,valore,@:
set_to_array_   nome,i,indice,valore,@:
set_to_matrix_  nome,i,riga,colonna,valore,@:
```

Il @ è un placeholder obbligatorio per il campo non usato (int o char).

Input / Output

```
print_ &i&nome&:           // stampa variabile int
print_ &s[]&nome&:         // stampa stringa intera
print_ [0]nome:           // stampa elemento 0 di un array (sintassi breve)
println_ &i&nome&:         // stampa e va a capo
lnprint_ &i&nome&:         // va a capo e poi stampa
lnprintln_ &i&nome&:       // va a capo, stampa, va a capo

print_ &s&"testo"&:        // stampa testo letterale
print_ &s&&:               // stampa uno spazio
```

Funzioni

```
od_ nomefunzione(argomento) {
    // corpo
    return_ &i&variabile&:
}
```

```
__start(){
    __nomefunzione(arg):           // chiamata senza valore di ritorno
    var1 = __nomefunzione(arg):    // chiamata con valore di ritorno
}
```

- `return_ NULL: 0 return_:` per funzioni void
- `return_ &i&nome&:` per restituire un valore

Blocco di sistema

```
#{
    debug_ -dt:           // attiva debug
    debug_ -df:           // disattiva debug
    exec_:                // avvia __start
}
```

Il blocco `#{ }` viene eseguito **prima** di `__start` e serve a configurare l'ambiente.

Condizioni e cicli (*implementazione in corso*)

```
if_( var1 == var2 ){    }
else_{    }
for_( var1 != var2, var1 + 1 ){    }
while_( var1 < var2 ){    }
```

La sintassi è già riconosciuta dal parser; l'esecuzione condizionale è in fase di sviluppo.

Sistema di debug

Il debug si attiva con `-dt` da riga di comando oppure con `debug_ -dt:` nel blocco `#{ }`.

Con il debug attivo, l'interprete stampa su stdout ogni passaggio interno:

- la riga analizzata da `parse()` con il suo indice
- ogni chiamata a `get_index()` con il token risolto
- ogni chiamata a `is_what()` con il tipo dedotto
- l'ingresso e l'uscita da ogni funzione tramite `exec_funarg()`
- la costruzione dello stato in `build_state()`
- la ricerca di funzioni in `is_function_()`

Esempio di output debug:

```
parse chiamato con global_ip: 5 e line_idx_program: 12
linea analizzata: 5 var1=42
[CHECK] var x n | left: var1 | right: 42
linea analizzata: 6 println_&i&var1&
DEBUG PRINT: type=i name=var1
get_index variabile var1 di tipo i
```

Il debug è pensato per tracciare problemi di parsing, errori di tipo e comportamenti inattesi senza dover usare un debugger esterno.

Test interni

Avviando l'interprete senza argomenti si esegue `run_test()`, che verifica:

- dichiarazione e scrittura di variabili `int`, `float`, `char`

- dichiarazione e scrittura di array int, float, char
- dichiarazione e scrittura di matrici int, float, char
- accesso tramite `get_index()` con indici numerici e variabili

./Zinterpreter

Output atteso:

```
LOG: decl_var ok
LOG: set_to_var ok
LOG: decl_array ok
...
=== VARIABILI ===
var0 (int)      : 3      [atteso: 3]
var1 (char)    : F      [atteso: F]
...
```

Stato del progetto

Funzionalità	Stato
Lettura e formattazione sorgente	☐
Variabili int / float / char	☐
Array int / float / char	☐
Matrici int / float / char	☐
Funzioni con argomenti e return	☐
Aritmetica + - * /	☐
Assegnazione = tra tutti i tipi	☐
Print / Println / Lnprint	☐
Sistema #{ } e debug	☐
If / Else	☐ in sviluppo
For / While	☐ in sviluppo
Scan (input utente)	☐ in sviluppo
Import librerie <code>.zlib</code>	⚠ deprecato (usa arg da CLI)

Licenza

Progetto personale — tutti i diritti riservati.